

Memory-based Customer Support Agent with Retrieval Evaluation

Yingjie Wu

Information Retrieval Assignment 2: AI Agents

Repository: <https://github.com/yingjie16330-spec/memory-support-agent>

Demo video: <https://youtu.be/B3CMyAvVKSE>

1 Introduction

This project implements a small customer support agent for a fictional SaaS product, CloudBox AI. The goal is to show that an AI system can improve its answers by retrieving information from external sources before generating a response, rather than relying on the language model’s internal knowledge alone. The agent retrieves from two sources: a product knowledge base and a per-user memory store. It is built as a *tool-calling agent*: the language model is given two retrieval tools and decides on its own which tool(s) to call, and with what query, before producing a grounded answer.

2 System Design

Instead of a fixed retrieve-then-answer sequence, the model is given two tools: `search_knowledge_base(query)`, which searches product documentation, and `search_user_memory(query)`, which searches the current user’s conversation history. The agent loop sends the user question and tool definitions to the model. The model decides which tools to call and what query to use for each. Tool results are fed back into the conversation, and the loop repeats (up to a small iteration limit) until the model produces a final answer. After the answer, the system runs a groundedness check and an escalation check based on what was actually retrieved. This separates retrieval from generation while still letting the model act as an agent that selects actions.

3 Information Retrieval Method

The retrieval method behind both tools is BM25, a classic and strong lexical IR baseline. It scores documents using term matching, term frequency, inverse document frequency, and document length normalization. BM25 was chosen over a basic TF-IDF cosine baseline because it is more standard and more suitable for a Master-level IR project. Dense retrieval was deliberately left out to keep the implementation small and interpretable; it is listed as future work.

4 Memory Component

The memory component stores synthetic conversation history for each user. Memory retrieval also uses BM25, but only over records belonging to the current `user_id`, making it user-specific. Its purpose is to add personal context and to avoid repeating advice the user already tried. For example, when asked about an API key, the model first called `search_user_memory`, saw that the user had already regenerated the key, and produced a next-step answer instead of repeating the earlier suggestion.

5 Evaluation Method

The project uses a synthetic test set of 20 questions with gold labels for relevant knowledge base documents (`gold_docs`), relevant user memory (`gold_memory`), and the escalation decision (`should_escalate`). Retrieval is evaluated with Hit@3, Precision@3, and Recall@3, appropriate because the agent retrieves a small top- k set before answering. The report also includes escalation accuracy and groundedness statistics. The groundedness check is a lightweight heuristic measuring lexical n-gram overlap between the answer and retrieved context after lowercasing, punctuation removal, and stopword removal. It is a hallucination-risk signal, not a proof of factual correctness. Because tool queries are LLM-generated, results vary slightly between runs; the numbers below come from one representative run.

6 Results

Evaluation over the 20-question test set (OpenAI gpt-4o-mini as the tool-calling model):

- Knowledge base Hit@3 = 0.85, Precision@3 = 0.35, Recall@3 = 0.83
- Memory Hit@3 = 0.79, Precision@3 = 0.26, Recall@3 = 0.79
- Escalation accuracy = 0.80
- Groundedness: 6 supported, 9 partially, 5 unsupported (avg 0.36)

Retrieval quality is good: the correct document is in the top 3 for 85% of questions. Precision@3 is low because most questions have only one or two gold documents, so the remaining top-3 slots count as non-relevant by construction; this is expected with a fixed $k = 3$ and a small gold set. The groundedness scores are low, but this reflects the limits of the heuristic rather than poor answers: because answers are generated in the model’s own words, they often paraphrase the source documents, so lexical overlap is low even when the answer is factually grounded. A semantic check (e.g. an LLM-as-judge) would be more accurate and is listed as future work.

7 Reflection on Using AI Coding Tools

AI coding tools were useful for speeding up implementation and drafting documentation, especially the CLI, test scaffolding, and the initial dataset. However, I treated the generated code as a draft, not as automatically correct. An early version implemented a fixed retrieve-then-answer pipeline and hard-coded some answers and escalation rules that happened to fit the test set. I identified this as a problem—it inflated the evaluation numbers and did not match the assignment’s requirement to build an agent—and rewrote the core so the model performs real tool calling, removing the hard-coded answers and test-specific rules. While reviewing the test set I also found one label (an EPUB question marked for escalation) that only existed to fit the old rule, and corrected it. The honest, lower numbers in this report come from the corrected version.

8 Limitations

The dataset is synthetic: real support data contains private information, so a small synthetic dataset was used for controlled, reproducible, inspectable evaluation. Other limitations: BM25 is purely lexical and may miss semantic matches; the memory store is small; escalation logic is rule-based (correct on 80% of test cases); the groundedness check underestimates paraphrased but correct answers; results vary between runs because tool queries are LLM-generated; and the system does not include dense retrieval, reranking, or long-horizon planning.